



A Cross-Platform Graphics API Solution for Modern and Legacy Development Styles

Robert Konrad^(✉) and Stefan Göbel

Serious Games Group, Technische Universität Darmstadt, Darmstadt, Germany
`robert@konradpost.de`, `stefan_peter.goebel@tu-darmstadt.de`

Abstract. The introduction of the Direct3D 12 and Vulkan graphics APIs has caused a big shift in how video games are developed. However, this shift has also brought about significant challenges, including increased complexity in graphics development and difficulty in supporting lower-end platforms like web browsers and older Android devices. As a result, many developers continue to rely on outdated APIs, despite their limitations.

To address this issue, this paper presents a novel software library that enables developers to seamlessly create applications using either a modern or a classic API style while simultaneously targeting all platforms, without additional effort. We provide a detailed analysis of the performance implications of each development strategy and effective tuning methods to optimize the library's performance.

Our approach is particularly important for serious games, as their target audience is used to the look of modern video games but does not necessarily own modern gaming hardware. By supporting lower-end devices, our software library enables developers to create serious games that can be accessed by a wider range of users, regardless of their device's age and processing power.

In summary, this paper offers a solution to the challenges associated with graphics development and cross-platform support, enabling developers to leverage the benefits of modern graphics APIs without sacrificing compatibility with lower-end devices.

Keywords: Cross-platform development · Graphics · GPGPU · Direct3D · Vulkan · Metal · OpenGL

1 Introduction

Serious Games like most software projects ideally run well on all widely used hardware and software platforms. But as with any video game using the graphics hardware efficiently across different platforms is a complex problem, requiring the use of complex and ever evolving APIs and incompatible, system-specific programming languages. The introduction of Direct3D 12 and Vulkan in 2015

and 2016 respectively has made the situation even more difficult overall as those APIs represent a big shift in how graphics APIs work but older APIs are still relevant as to this day the new APIs cannot be used everywhere.

The most common solution to this problem is to use a game engine that besides providing a big number of various features for game development also solves the portability problem. But even the best-known game engines, developed by teams of hundreds of developers, struggle with supporting all platforms with Unreal deprecating support for web-browsers and Unity not officially supporting mobile browsers¹. Serious Games in particular can have technical or market requirements that do not align with the priorities in the development of game engines which generally focus resources on supporting pure entertainment games.

Kore in C (in short Kinc)² is a development toolkit for developing games and other multimedia applications on the lowest possible level while not compromising on portability. Notably portability reaches further than what today's game engines typically provide. Kinc aims to provide an alternative for developers who prefer to have bigger control over the details of their software projects. As such it is not specifically aimed at Serious Games but also does not restrict itself to the necessities of common Entertainment Games, providing the options to optimize for more specialized needs like very broad portability or very small size.

The paper details work done for the Kinc project and in particular focuses on the aspects of providing portability across old and new graphics APIs (henceforth called legacy and modern APIs) and details the problem space from a technical perspective and from a market perspective.

2 Related Work

The actual graphics APIs that are in active use today are Direct3D 11 and 12, OpenGL and variations thereof like OpenGL ES and WebGL, Vulkan, Metal and WebGPU. The relevant intricacies of each API are discussed in a later chapter in terms of how they directly relate to the presented work it is of note that each of them does try or has tried to solve the cross-platform problem by itself but as of now each API is only available or updated on a subset of today's relevant software and hardware platforms which unfortunately transforms them from a solution into problems.

Every multi-platform game engine is confronted with the problem of targeting multiple graphics APIs. This problem is typically handled in the very core of a game engine (hence the name of our toolkit) with all other functionality being layered on top of it. Game engines can however allow themselves to be sloppy in this work as this code only has to support the feature set of one engine and it is not a serious problem when some aspects of handling the differences between the graphics APIs leak into higher levels of the codebase.

¹ <https://docs.unity3d.com/Manual/webgl-browsercompatibility.html>.

² <https://github.com/Kode/Kinc>.

Many other projects do exist that try to directly solve the problem of cross-platform graphics programming, the most well-known of which are bgfx³, gfx-rs⁴ and sokol⁵.

Similar projects but with a more specific focus of mapping existing graphics-APIs to each other are Google’s ANGLE, Microsoft’s D3D11On12 and MoltenVK. These projects implement a legacy graphics API on top of another legacy graphics API, a legacy graphics API on top of a modern graphics API and a modern graphics API on top of another modern graphics API. Notably absent however is an implementation of a legacy graphics API on top of a modern graphics API, which is a focus of this paper.

3 Serious Games and the Software/Hardware Ecosystem

Big video game productions primarily or even exclusively use specialized distribution channels that were setup specifically for games. PC games are sold on platforms like Steam and GOG and video game systems like a PlayStation 5 can be seen as a video game distribution platform in itself. While some platforms like Steam have opened up over the years, most still require a certification process that companies have to go through for any game they want to release. These processes are not necessarily open for Serious Games as properties like financial viability and technical experience with video game consoles are assessed. This is just one example of how distribution of Serious Games differs compared to the distribution of the latest Call of Duty.

Smaller productions often concentrate on distribution platforms like Google Play, Apple’s App Store and web browsers. This opens up a much broader market but the hardware and software eco-system is much less mature with very old hardware still being in use and graphics drivers not necessarily being up to date. These issues can be even more pressing when software is not targeted at gamers which are used to handling hardware and driver problems on their side to some extent.

Targeting older hardware poses specific challenges in graphics programming. The chips that drive the graphics (commonly called GPUs for Graphics Processing Units) vary greatly in how the individual hardware works and outside of game console programming a driver layer runs on top of it that tries to unify and abstract how different GPUs are accessed. Abstractions however tend to leak details of underlying layers in various ways and for graphics APIs performance characteristics in particular are leaking which can be characteristics of the hardware or the driver or some combination of both. An interesting side-effect that can be witnessed based on this is that Nvidia and AMD regularly release driver updates to concede with big game releases to provide specific optimizations for those titles. The driver packages are also getting surprisingly complex with Nvidia’s current driver package being bigger in size than Windows XP.

³ <https://github.com/bkaradzic/bgfx>.

⁴ <http://gfx-rs.github.io>.

⁵ <https://github.com/flooh/sokol>.

Graphics API design advanced to better handle this situation and in particular about eight years ago a cut across the industry started to happen that fundamentally shuffled around how GPU APIs work and in 2023 this finally also arrived in web browsers via the new WebGPU API.

While big parts of the games industry can now ignore older graphics APIs as they can expect their customers to use fairly up to date hardware and drivers or even dedicated gaming devices, Serious Games still have many years ahead of them during which they will have to support old graphics APIs so they can reach a large enough audience - especially with distribution channels for higher end systems often being cut off.

3.1 The Software Ecosystem

In 2013 AMD introduced a new graphics API called Mantle that specifically was not aimed at adding new features but to provide a more efficient programming model with the intention of making drivers simpler and achieving more reliable performance. This called a ripple effect in the industry with Metal in 2014, Direct3D 12 in 2015, Vulkan in 2016 and finally WebGPU in 2023 all following a very similar approach.

These new APIs require relatively recent hardware and they also require updated graphics drivers. Driver updates though are often unnecessarily restricted in various ways. In Android, iOS and macOS drivers are bundled with the operating system and only recent hardware receives operating system updates, especially for Android devices. On iOS even web browser updates are restricted by operating system updates, further slowing down the adoption of new features on top of all the other factors that already caused the release of WebGPU to happen seven years after the release of Vulkan.

All of this created a market that is currently split between the new graphics APIs and the old graphics APIs and all of the following APIs are still in wide use.

Direct3D 10/11

Introduced in 2006 and 2009 with Windows Vista and Windows 7 respectively, those new versions of Direct3D were a major rework of earlier versions and were only supported by Windows Vista and later [1].

Notably Direct3D 11 introduced support for compute shaders. It also added support for tessellation shaders after Direct3D 10 already added support for geometry shaders. Support for these shader stages was not widely adapted and never implemented for Apple's mobile devices. Geometry and tessellation shaders have now been supplanted by mesh shaders in Direct3D 12, Vulkan and Metal.

As discussed in Sect. 4, Direct3D 10 stopped supporting free changes to rendering state and instead supported a set of state objects that can be created and made active.

Direct3D 12

Direct3D 12 was another major change of the API that switched it from a legacy style API to modern style API. Microsoft also provides its own implementation of a legacy API on top of a modern API with 11on12 but not for the opposite.

Metal

Metal is a graphics API introduced by Apple solely for iOS and macOS [2]. While meanwhile support for intel and AMD was added after Apple also moved to exclusively using their own GPUs for Macs, with Apple switching all of their devices to use their own graphics hardware, Metal can now be seen as an API just for one series of hardware, in that sense being quite similar to how graphics APIs on consoles work.

OpenGL

OpenGL is an API that goes back many years, being introduced in 1992 - and even further as the initial version of OpenGL was based on an even earlier API called IrisGL [3]. Major changes were introduced in the early 2000s when OpenGL moved to utilize programmable shader pipelines. As of now development of OpenGL has mostly stopped despite initial intentions of keeping it around as an easier to use alternative to Vulkan.

OpenGL ES

OpenGL ES is a variation of OpenGL for smaller devices with slight changes to the APIs and its own version numbers [4]. The interface fundamentally stayed the same but several restrictions were introduced as well as support for lower precision data types. While further development on OpenGL ES stopped, it continues to be available on all Android devices. For iOS on the other hand OpenGL ES was deprecated - notably long before its last version so that features added to later versions of OpenGL ES like support for compute shaders never arrived on iOS.

WebGL

WebGL is closely based on OpenGL ES with WebGL 1 being based on OpenGL ES 2 and WebGL 2 being based on OpenGL ES 3.

In the Windows versions of Chrome and Firefox the implementation of WebGL is based on ANGLE, itself being an implementation of OpenGL on top of Direct3D 9 and 11. Together Chrome and Firefox represent a very high percentage of the browser market on Windows, Linux and Android as most other browsers are based on Chrome, which even includes new versions of Microsoft Edge.

Chrome and Firefox are also widely used on macOS but its default browser is Safari. On iOS only Safari is important as other browser implementations are prevented by disallowing the code execution of runtime-allocated memory - a fundamental feature for fast JavaScript execution using just in time compilation. While programs called “Google Chrome” and “Mozilla Firefox” do exist on iOS, they are just UIs on top of the Safari browser engine. This unfortunately restricts iOS to the Safari feature set, that is often lacking behind many years.

Vulkan

AMD introduced a new GPU API called Mantle in 2013. Notably AMD provided the GPUs of most of the game consoles of the time, therefore having experience with providing custom GPU APIs that are actually very widely used. Vulkan was built based on Mantle but adjusted to run on non-AMD GPUs as well. It became a formalized standard in 2016 and Vulkan drivers are available for Windows, Linux and Android [5]. Vulkan and Direct3D 12 are surprisingly similar with the main difference being that Vulkan supports render passes, which are important for deferred rendering hardware that is widely used in mobile hardware to reduce the reliance on fast memory.

WebGPU

WebGPU is a new API that as of 2023 has just had its initial public release with the latest Windows version of Google Chrome. Work for supporting WebGPU on more platforms in Google Chrome is of course underway as is work for support in Firefox and Safari. While the work in Firefox seems to be far along it is questionable when support in Safari will arrive - basing predictions on Apple's track-record with WebGL it would still be many years away, potentially being a major hindrance in adopting new graphics APIs. WebGPU is in its feature set similar to the initial versions of Metal, Vulkan and Direct3D 12 as it is designed to also work on top of exactly those [6].

Video Game Consoles

Consoles classically provide their very own graphics APIs. It is a very different problem space as only a single piece of hardware needs to be supported, making a driver ecosystem unnecessary and allowing the distribution of low-level software updates as part of the games. The specifics of any of these console specific APIs are unfortunately confidential.

3.2 Market Situation

As of May 2023, the Android Dashboard⁶, which collects data from devices accessing Google Play, which is Google's online store for Android devices, shows that 15% of Android cannot yet use Vulkan which can consequently be a very significant percentage of a target group that is less likely to regularly update their tech gadgets than the overall average.

Concerning browsers, Google Chrome is automatically kept up to date on Android devices. As of now Google Chrome for Android does not support WebGPU but once it does, likely a large percentage of Android devices will quickly get access to it.

If the browser situation on Android develops as positively as expected in terms of WebGPU support, this will likely represent a situation that is reversed to what can be found for Apple's iOS devices. On the system API level, the Metal API that was introduced in 2014 is now very widely available with statistics by Roblox from December 2021 seeing it at 99,9%⁷. On the other hand, Apple

⁶ <https://developer.android.com/about/dashboards>.

⁷ <https://gist.github.com/zeux/cc7be14916e36c51eb56fb7ed372206b>.

tends to lack far behind in supporting the latest features in its Safari browser. WebGL 2 was first supported in Safari 15, which released in September 2021, compared to Chrome which supports it since January 2017. Interpolating from here WebGPU support in Safari can possibly be expected in 2027 or 2028.

The situation is not ideal on larger devices either, especially with many laptops not allowing installs of up-to-date graphics drivers and very old devices still being in use as old computers can still be fast enough for many use cases. Steam lists only 2023 of PCs as being able to run Direct3D 12 in their April 2023 hardware survey⁸.

4 Efficient and Portable Graphics Implementation

Basing the low-level graphics abstraction can be based on modern or legacy APIs and as long as both kinds of APIs have to be supported both strategies have varying upsides and downsides. Following up is a look at the major differences and the consequences of either strategy for each of these differences.

4.1 Efficiency Features

The interesting aspect of the new graphics APIs in terms of cross-platform support is that the primary features are not concerned with adding new functionality but with improving efficiency and reliably of performance characteristics. Following up is a closer look at the most important of those features with a discussion of how each of them can be mapped from a modern API to a legacy API and vice versa.

4.2 Uniform Data

Graphics-APIs provide a plethora of options for binding data that is constant across draw calls. The actual process of binding data used to consist of querying individual variable names and then setting the data using one call for each variable. Later versions of Direct3D and OpenGL added the option to update a whole structure of variables all at once. Modern APIs always work like that and therefore all APIs are fundamentally aligned in how uniform data can be updated - unless support for WebGL 1 is still required to support older Apple devices. In this case a loop through a data struct can be used to set the struct values variable by variable. The opposite procedure is also possible as partial data-updates can be used to update singular variables inside of a struct in newer APIs.

More advanced features of uniform data handling in modern graphics APIs are directly linked to its handling of data-synchronization and are therefore discussed in the following sub-section.

⁸ <https://store.steampowered.com/hwsurvey/Steam-Hardware-Software-Survey-Welcome-to-Steam>.

4.3 Data-Synchronization

In legacy graphics APIs synchronization of data-updates between the CPU and the GPU is handled automatically. When writing to a vertex-buffer from the CPU while it is still in use by the GPU, the driver will apply a strategy to hide that problem and allow the writes. Typically, it will either create a shadow-copy of the vertex-buffer which has the downside of taking up more memory or it will stall until the GPU finishes the work which is of course detrimental to performance. Drivers commonly employ custom heuristics to decide on the specific strategy. Unfortunately, those heuristics are hidden from the programmer and can vary wildly between different GPUs and graphics drivers which can make it hard to optimize code to run fast on all systems. This is also one of the heuristics that driver vendors typically tweak specifically for important applications.

In modern APIs this process is instead entirely manual which makes it harder to implement data-handling initially but makes it much easier to optimize it for all platforms as the performance characteristics are reliable and do not require testing on a big collection of different hardware [7].

When running a legacy API on top of a modern API, strategies very similar to what legacy API drivers do have to be employed. An advantage compared to drivers is that application authors can tweak values manually to adjust them to the needs of the application - like a value for how many shader-copies to create maximally and whether those copies should be created initially or on demand. Essentially application authors can build in the tweaks by themselves that driver authors build in for the applications they consider important.

Running a modern API on top of a legacy API is trivial for data-synchronization - synchronization-functions are simply not implemented. This results in a situation very similar to programming a legacy API directly but the advantages of using a modern API unfortunately disappear.

Some modern graphics APIs also support more advanced features like embedding small amounts of data in the GPU command-stream, thereby automatically synchronizing those data-updates with other GPU-commands. This cannot be directly implemented in legacy APIs and needs to be emulated using regular data-binding methods. This is a potential performance pitfall as there is no way to tell when a driver can optimize data-updates using similar methods.

4.4 Pipeline State Objects

In OpenGL every piece of rendering state is setup with a separate function call. This is on the one hand error prone because it is easy to forget that one particular piece of rendering state was still active and possibly set to an unfortunate value. On the other hand, it is easy to use and very flexible - if only one piece of state needs to change between draw calls this results in one function call and consequently the changes required to be executed by the driver can also be very small. However, this is not always the case and a single state-change can actually require a recompilation of all shaders used in following draw calls. It is up to the hardware vendor to decide how different pieces of rendering state are

implemented and they can decide to make it a switch or integrate it into the shader instructions. The dynamic nature of rendering state in OpenGL has led to the situation that drivers try to defer a lot of work until actual draw-calls are executed because only then all information has to be present that might be needed to decide how to exactly compile the bound shaders.

All modern APIs use the concept of pipeline state objects which sum up all rendering state, combined with a set of shaders and the necessary input layout for the shaders, into a single object which is then compiled. This was done to allow application developers to specify themselves when any shader compilation takes place. Notably though this has more recently come under scrutiny as it can lead to very big amounts of PSOs in big projects that take a very long time to compile. An extension for Vulkan was recently published that again allows to change rendering-state piece by piece.

Using a PSO-API on top of singular state-changes is simple, as when making a PSO active the implementation can directly iterate and set all state. This procedure isn't even as inefficient as it may appear because as mentioned earlier, legacy APIs tend to defer actual work until a draw-call happens.

When running an OpenGL style on top of a modern API the situation becomes however very complicated and would re-introduce the same random compiles so known to modern APIs. Direct3D 11 though already uses an API that is very close to PSOs, just splitting it up into several state collection objects. For these reasons Kinc uses PSOs for both its legacy and its modern API.

4.5 Multithreaded Command Submission

A fundamental feature of modern GPU APIs is better support for utilizing multi-core CPUs, primarily by means of recording GPU-commands simultaneously from multiple threads. On legacy APIs commands for the GPU are generated directly via function calls ala `drawTriangle` and these calls do not tend to be thread-safe. Modern APIs instead provide a structure representing a list of commands and multiple of those lists can be recorded and prepared from multiple threads. Finally, they are stitched together on a single thread and submitted but the bunch of the work can be distributed across multiple cores this way - and work has to be done because all of the commands have to be converted into data-structure that can be transferred to the GPU and executed by its command-processor but the final submission of these commands is then a relatively simple task.

When implementing a legacy API on top of a modern API this can simply work on top of a single command-list. A second command list can often be useful for implementing features that are implicit in legacy APIs independently of the primary list, like the transfer of data from CPU memory to GPU memory or the handling of framebuffer-flips. Care has to be taken for when to submit the command-list as it has to work together with the mechanisms for synchronizing data to be efficient. Usually, it will make sense to submit the command-list directly after every draw-command but it can of course be tweaked further for the application at hand.

When implementing a modern API on top of a legacy API clearly the actual GPU-API calls have to happen from a single thread as required by the API. A data-structure has to be implemented to collect commands and a procedure that iterates over that structure and does actual graphics-API-calls based on the collected data. This is clearly overhead compared to simply doing the calls directly but unavoidable to serve applications which make use of the threading-features. The overhead can however be minimal if the application is otherwise efficient in how its command-submission works. If that is the case the multi-threaded command collection will run quickly as it only has to do minimal work on the graphics-API-side. The actual command-calls can also happen with little overhead as traversing a tightly packed array of command-structures is quick compared to doing any actual driver-work.

4.6 Summary

When purely looking at the performance features, using a legacy API or a modern API is both viable, no matter what APIs a program finally has to work on and both strategies can be used without major performance problems on top of the performance problems inherent to legacy APIs. If the programming team is comfortable with it, it is consequently advisable to already use a modern API even if it has to run on top of a legacy API. It does not pose a performance regression and it provides new options once legacy APIs become less important.

4.7 Functionality Features

While it is true that the new GPU APIs were primarily focused on efficiency in their design a lot of features have also been added that cannot feasibly be worked around on older APIs and have to be considered.

4.8 Compute Shaders

Compute shaders also have relatively general performance advantages by providing a local cache and making it easier for GPUs to schedule the work with requirements for how to work has to be grouped being relaxed for example thanks to the absence of instructions requiring derivatives [8]. Compute shaders can also be scheduled to run in parallel to any other work while other shader-types do their work on after another.

Compute shaders were already implemented in Direct3D 11 and OpenGL but are noticeably absent in all versions of WebGL and in the latest versions of OpenGL ES supported on iOS.

4.9 Bindless Resource Access

Modern APIs allow shaders to select themselves what textures or other GPU-objects to use dynamically, which can simplify and speed up rendering [9,10].

This functionality unfortunately cannot be emulated and is in fact not even available in all systems that support Vulkan and is therefore only exposed as an extension. On the other hand, an Nvidia-extension for OpenGL is available [11]. A possible workaround to using bindless access are big lists of if-else-constructions but the number of textures that can be bound classically is very restricted and if-else-constructs can lead to large amounts of textures being sampled even when only one is ultimately used.

4.10 Raytracing

Going beyond just new APIs, raytracing is now supported in hardware on some GPUs and the functionality is exposed in Direct3D 12 and Vulkan. Modern GPUs also provide general features to make raytracing more feasible performance-wise, notably some of the very latest models avoid divergence in shader-execution by dynamically re-bundling shader-waves. Compute shader-based implementations do exist [12]. Notably CryEngine supports Raytracing in compute shaders as do the drivers for Nvidia's 10x0 series of GPUs. In the latter case this can directly be compared to the hardware implementation of the following generation of graphics chips. It can be feasible for smaller use-cases but it has to be kept in mind that the developers at CryTek and Nvidia can likely optimize a raytracing implementation much further than most developers would be able to and with the big performance jumps seen in the latest two generations of Nvidia's hardware following that (the 30x0 and 40x0 series) there is doubt in the comparable viability of computer shader implementations even when taken care of by the biggest experts in the field.

4.11 Mesh Shaders

Mesh shaders are a new concept that is intended to replace geometry shaders and tessellation shaders, roughly based on work that was done to build geometry pipelines using compute shaders [13, 14]. Notably support was recently added to Metal and Apple's GPUs - neither of which ever supported geometry or tessellation shaders. Wide support for older devices is many years away but based on the earlier work to use compute shaders for a similar implementation, more widely usable implementations are feasible. A major hindrance is however the web-platform as it only just now gained support for compute shaders via WebGPU.

5 Conclusion

Modern graphics APIs make development easier for big and experienced development teams by providing more reliable performance which is a key aspect for a robust and fun game release. When concentrating on up-to-date hardware these teams can fully focus on working with these APIs. But smaller or more specialized teams as they can commonly be encountered in the Serious Games industry have a need to target a wider market, in particular to reach non-gamers. In this

space older graphics APIs are still relevant and the overall fragmentation into very different APIs poses a technical challenge. Our paper detailed solutions to these problems and the pitfalls that need to be avoided.

The Kinc project which this paper is based on already provides implementations of all the aforementioned aspects, is open source and is actively used on all of today's GPU-supported platforms, from mobile web browsers to video game consoles. It is successfully used in video games such as CrossCode⁹ as well as video game development tools like Armory3D¹⁰ or RPG Playground¹¹.

Portable graphics programming is an ever-expanding problem space as long as the graphics APIs themselves are ever expending. On the other hand, some problems disappear with old hardware becoming obsolete and getting replaced. As such further work will be needed as long as this is the case and the second part of the fourth chapter can also be seen as a preview of what aspects of graphics APIs might require more work to facilitate high portability of graphics code in the future.

Acknowledgements. The investigated research of this contribution partially has been funded by strategic invest of the Serious Games group at TU Darmstadt in the context of its research activities for the creation/authoring of Serious Games.

References

1. Blythe, D.: The direct3D 10 system. In: ACM SIGGRAPH 2006 Papers, pp. 724–734 (2006)
2. Sandmel, J.: Working with metal-overview. Apple WWDC (2014). <https://developer.apple.com/videos/wwdc/2014>
3. Frazier, C., Leech, J., Brown, P.: The OpenGL graphics system: a specification (2016)
4. Munshi, A., Leech, J.: OpenGL ES common profile specification version 2.0.25 (2010)
5. The Khronos Vulkan Working Group: Vulkan 1.0.48 - A Specification (2017)
6. Kenwright, B.: Introduction to the WebGPU API. In: ACM SIGGRAPH 2022 Courses, pp. 1–184 (2022)
7. Lustig, D., Martonosi, M.: Reducing GPU offload latency via fine-grained CPU-GPU synchronization. In: 2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA), pp. 354–365. IEEE (2013)
8. Bilodeau, B.: Efficient compute shader programming. In: Presentation, Game Developers Conference, San Francisco, CA (2011)
9. Pettineo, M.: Using bindless resources with DirectX raytracing. In: Marrs, A., Shirley, P., Wald, I. (eds.) Ray Tracing Gems II, pp. 257–279. Apress, Berkeley (2021). https://doi.org/10.1007/978-1-4842-7185-8_17
10. Abdellah, M., Eldeib, A., Owis, M.I.: GPU acceleration for digitally reconstructed radiographs using bindless texture objects and CUDA/OpenGL interoperability. In: 2015 37th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC), pp. 4242–4245. IEEE (2015)

⁹ <http://www.cross-code.com>.

¹⁰ <https://armory3d.org>.

¹¹ <https://rpgplayground.com>.

11. Bolz, J.: OpenGL bindless extensions. NVIDIA Corporation (2009)
12. García, A., Ávila, F., Murguía, S., Reyes, L.: Interactive ray tracing using the compute shader in DirectX 11. In: GPU Pro 360, pp. 117–140. AK Peters/CRC Press (2018)
13. Carvalho, M.Â.A.D.: Exploring mesh shaders. Doctoral dissertation (2022)
14. Peddie, J.: The sixth era GPUs: ray tracing and mesh shaders. In: Peddie, J. (ed.) The History of the GPU-New Developments, pp. 323–360. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-14047-1_7